

MeloStream

Plateforme de streaming musical

Rapport de projet

Application web full-stack avec Spring Boot, Angular et MySQL

Réalisé par : Ayoub

Type : Projet web full-stack

Backend : Java 21, Spring Boot

Frontend : Angular, TypeScript

Base de données : MySQL

Année universitaire : 2025–2027

30 mai 2026

Table des matières

Introduction	3
Contexte	3
Objectifs	3
1 Cahier des charges	4
1.1 Problématique	4
1.2 Besoins identifiés	4
1.2.1 Besoins fonctionnels	4
1.2.2 Besoins non fonctionnels	5
1.3 Solutions proposées	5
2 Conception fonctionnelle	6
2.1 Acteurs	6
2.2 Diagramme de cas d'utilisation	7
2.3 Diagramme de classes	8
2.4 Scénario de partage d'un titre	8
3 Conception technique	9
3.1 Architecture globale	9
3.2 Technologies utilisées	9
3.3 Structure du projet	10
3.4 Sécurité	10
3.5 Endpoints REST principaux	10
4 Réalisation	12
4.1 Fonctionnalités développées	12
4.1.1 Authentification	12
4.1.2 Recherche musicale	12
4.1.3 Lecture	12

4.1.4	Favoris	12
4.1.5	Playlists	12
4.1.6	Partage	13
4.1.7	Administration	13
4.2	Captures d'écran	13
4.3	Comptes de démonstration	17
5	Installation et lancement	19
5.1	Prérequis	19
5.2	Configuration de la base	19
5.3	Lancement du backend	19
5.4	Lancement du frontend	19
5.5	Activation optionnelle de Jamendo	20
6	Vérification	21
6.1	Tests backend	21
6.2	Tests frontend	21
6.3	Build frontend	21
	Conclusion	22

Introduction

Contexte

Les plateformes musicales occupent aujourd’hui une place importante dans les usages numériques. Elles permettent de rechercher des chansons, découvrir des artistes, organiser ses préférences et partager facilement des titres avec d’autres utilisateurs.

Le projet MeloStream consiste à développer une application web de streaming musical dans un contexte pédagogique. L’application propose une interface moderne côté client, une API REST sécurisée côté serveur et une base de données permettant de conserver les utilisateurs, les sessions et les titres favoris.

Le système utilise des services musicaux externes pour récupérer les chansons, les artistes, les albums, les pochettes et les liens audio. Deezer est utilisé par défaut pour les extraits audio, tandis que Jamendo peut être activé afin de proposer des titres libres ou licenciés.

Objectifs

Les objectifs du projet sont :

- créer une application web complète avec une séparation claire entre frontend et backend ;
- permettre l’inscription, la connexion, la déconnexion et la gestion du profil utilisateur ;
- proposer un catalogue musical consultable par recherche ou par humeur ;
- intégrer un lecteur audio sans lancer automatiquement la musique après la connexion ;
- permettre la sauvegarde des titres favoris ;
- permettre la création de playlists et l’organisation des titres ;
- ajouter une fonctionnalité de partage de titre par lien ;
- fournir une interface d’administration protégée par rôle ;
- produire une documentation claire pour comprendre, lancer et maintenir le projet.

Chapitre 1

Cahier des charges

1.1 Problématique

La problématique principale est de concevoir une plateforme musicale simple, ergonomique et sécurisée, capable de relier une interface Angular à une API Spring Boot et à une base de données MySQL. Le projet doit aussi communiquer avec des API musicales externes sans rendre le code difficile à maintenir.

L'application doit donc répondre à trois exigences majeures :

- fournir une expérience utilisateur fluide ;
- protéger les fonctionnalités privées par authentification ;
- garder une architecture technique claire et évolutive.

1.2 Besoins identifiés

1.2.1 Besoins fonctionnels

Référence	Description
BF01	Créer un compte utilisateur.
BF02	Se connecter et se déconnecter.
BF03	Consulter et modifier son profil.
BF04	Rechercher des titres par mot-clé, artiste, album ou humeur.
BF05	Écouter un titre depuis un lecteur intégré.
BF06	Ajouter un titre aux favoris.
BF07	Supprimer un titre des favoris.
BF08	Créer, renommer et supprimer une playlist.
BF09	Ajouter ou retirer des titres d'une playlist.
BF10	Partager un titre avec un lien direct.
BF11	Charger automatiquement le titre ciblé lorsqu'un lien partagé est ouvert.

BF12	Accéder à un espace administrateur pour gérer les utilisateurs.
------	---

1.2.2 Besoins non fonctionnels

Référence	Description
BNF01	L'interface doit être responsive et lisible.
BNF02	Les mots de passe doivent être stockés sous forme de hash.
BNF03	Les endpoints privés doivent être protégés par jeton.
BNF04	Les rôles doivent limiter l'accès à l'administration.
BNF05	Le code doit être organisé par domaine fonctionnel.
BNF06	La base de données doit être initialisée automatiquement en local.
BNF07	Le projet doit être testable et documenté.

1.3 Solutions proposées

La solution proposée repose sur :

- une application Angular pour l'interface utilisateur ;
- une API Spring Boot pour la logique métier et la sécurité ;
- une base MySQL pour la persistance ;
- une intégration Deezer/Jamendo pour les données musicales ;
- une authentification stateless par jeton ;
- une interface administrateur accessible seulement au rôle **ADMIN**.

Chapitre 2

Conception fonctionnelle

2.1 Acteurs

Acteur	Rôle dans le système
Visiteur	Peut créer un compte ou se connecter.
Utilisateur	Peut rechercher, écouter, sauvegarder, organiser et partager des titres.
Administrateur	Peut gérer les utilisateurs et consulter les statistiques.
API musicale externe	Fournit les titres, artistes, albums, images et audios.

2.2 Diagramme de cas d'utilisation

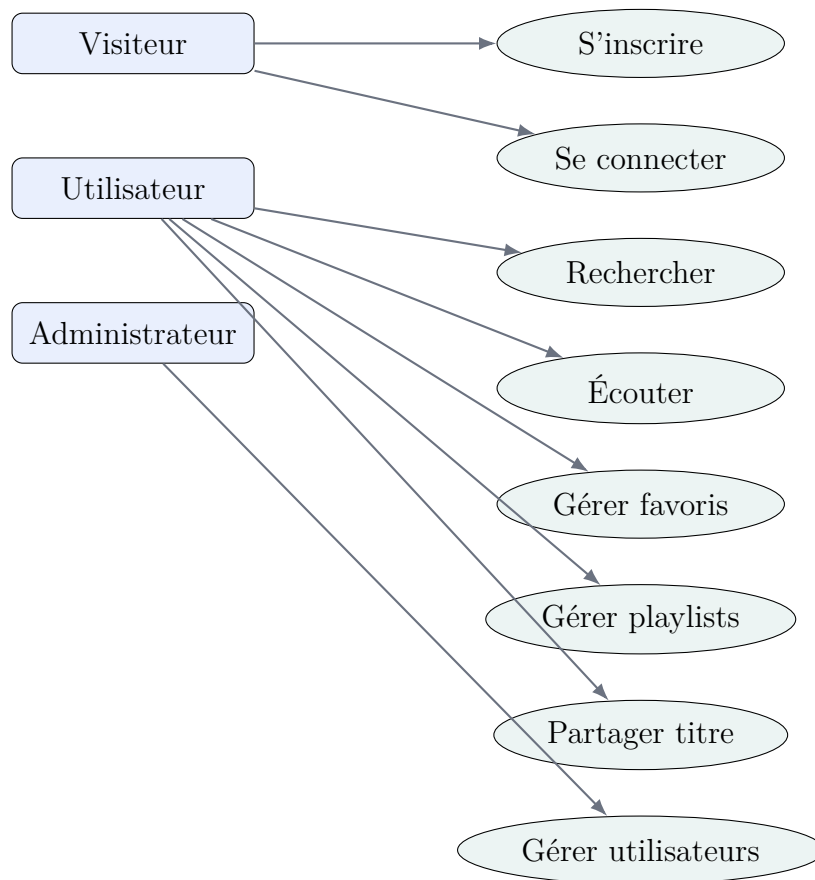


FIGURE 2.1 – Diagramme de cas d'utilisation simplifié

2.3 Diagramme de classes

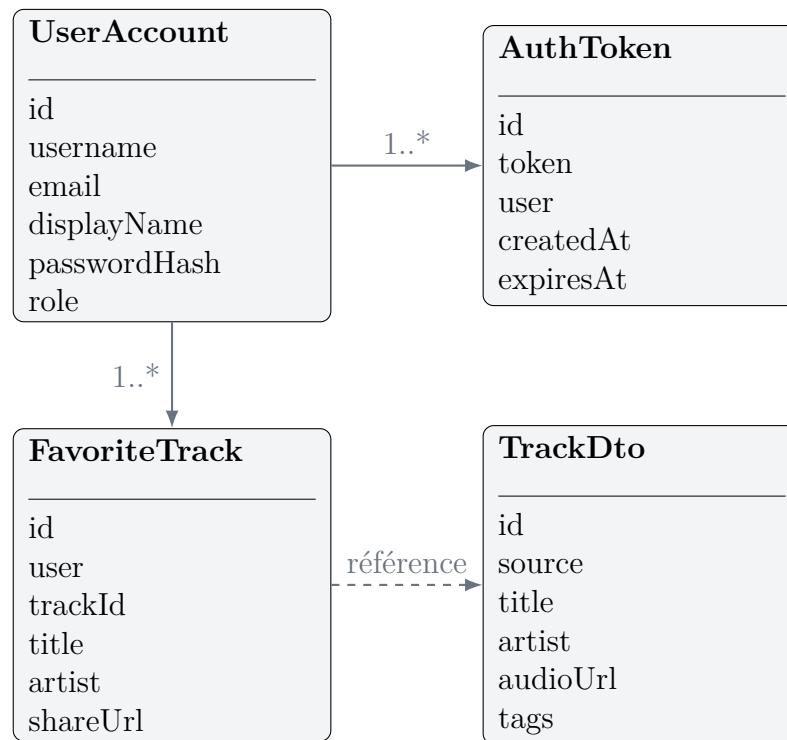


FIGURE 2.2 – Diagramme de classes principal

Le modèle est complété par les classes **Playlist** et **PlaylistTrack**. Une playlist appartient à un utilisateur, et une playlist contient plusieurs titres. Cette relation permet d'organiser les chansons au delà de la simple liste de favoris.

2.4 Scénario de partage d'un titre

Le partage est une fonctionnalité importante de l'application. Elle permet à un utilisateur de copier un lien vers un titre et à un autre utilisateur d'accéder directement au même titre.

1. L'utilisateur clique sur le bouton **Partager**.
2. Le frontend construit une URL au format `?song=source:id`.
3. Exemple : <http://localhost:4200/?song=deezer:3135556>.
4. Un autre utilisateur ouvre ce lien.
5. Après connexion, le frontend appelle l'API `/api/tracks/{source}/{trackId}`.
6. Le titre est chargé dans la zone de lecture rapide.
7. La musique ne démarre pas automatiquement.

Chapitre 3

Conception technique

3.1 Architecture globale

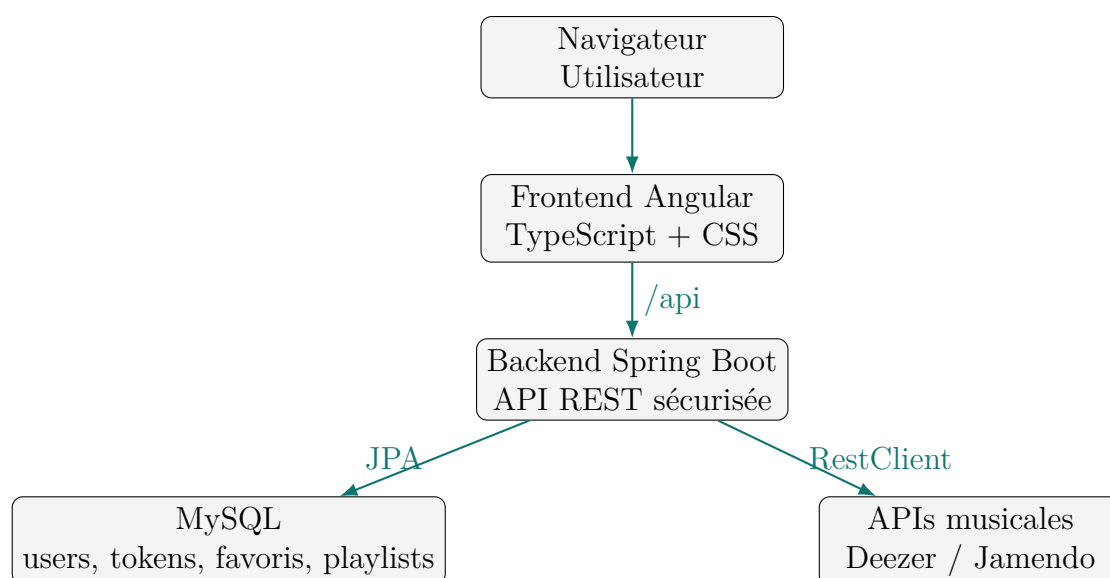


FIGURE 3.1 – Architecture technique de MeloStream

3.2 Technologies utilisées

Couche	Technologies
Frontend	Angular 21, TypeScript, RxJS, CSS responsive, @lucide/angular.
Backend	Java 21, Spring Boot 4.0.6, Spring Web MVC, Spring Security, Spring Data JPA.
Base de données	MySQL avec accès possible via phpMyAdmin.
APIs externes	Deezer public API et Jamendo API optionnelle.
Tests	Spring Boot Test, H2 pour les tests backend, Angular/- Vitest côté frontend.

3.3 Structure du projet

```
project web/  
  backend/  
    src/main/java/com/musicstream/  
      admin/  
      auth/  
      config/  
      deezer/  
      favorites/  
      jamendo/  
      music/  
      security/  
    src/main/resources/application.yml  
  pom.xml  
  frontend/  
    src/app/  
      admin-api.ts  
      app.css  
      app.html  
      app.spec.ts  
      app.ts  
      auth-api.ts  
      favorite-api.ts  
      music-api.ts  
    package.json  
    proxy.conf.json  
  rapport/  
    rapport.tex  
    rapport.pdf  
  README.md
```

3.4 Sécurité

La sécurité est assurée par Spring Security. L'application utilise un modèle stateless : après connexion, le backend génère un jeton stocké en base dans la table `auth_tokens`. Le frontend transmet ensuite ce jeton pour accéder aux endpoints protégés.

- `/api/auth/login` et `/api/auth/register` sont publics.
- Les endpoints `/api/**` nécessitent une authentification.
- Les endpoints `/api/admin/**` nécessitent le rôle `ADMIN`.
- Les mots de passe sont hashés avant stockage.
- Les règles CORS autorisent le frontend local sur le port 4200.

3.5 Endpoints REST principaux

Méthode	Endpoint	Description
POST	/api/auth/register	Créer un compte.
POST	/api/auth/login	Se connecter.
GET	/api/auth/me	Récupérer l'utilisateur connecté.
PUT	/api/auth/profile	Modifier le profil.
POST	/api/auth/logout	Se déconnecter.
GET	/api/tracks	Rechercher des titres.
GET	/api/tracks/{source}/{trackId}	Charger un titre partagé.
GET	/api/favorites	Lister les favoris.
POST	/api/favorites	Ajouter un favori.
DELETE	/api/favorites/{trackId}	Supprimer un favori.
GET	/api/playlists	Lister les playlists.
POST	/api/playlists	Créer une playlist.
PUT	/api/playlists/{playlistId}	Renommer une playlist.
DELETE	/api/playlists/{playlistId}	Supprimer une playlist.
POST	/api/playlists/{playlistId}/tracks	Ajouter un titre à une playlist.
DELETE	/api/playlists/{playlistId}/tracks/{source}/{trackId}	Retirer un titre d'une playlist.
GET	/api/admin/stats	Consulter les statistiques.
GET	/api/admin/users	Lister les utilisateurs.
PUT	/api/admin/users/{userId}/role	Modifier un rôle.
DELETE	/api/admin/users/{userId}	Supprimer un utilisateur.

Chapitre 4

Réalisation

4.1 Fonctionnalités développées

4.1.1 Authentification

L'utilisateur peut créer un compte, se connecter et se déconnecter. Après une connexion réussie, un jeton est utilisé pour sécuriser les requêtes suivantes. Les comptes de démonstration sont créés automatiquement au démarrage.

4.1.2 Recherche musicale

Le catalogue permet de rechercher des titres par mot-clé. L'utilisateur peut chercher un titre, un artiste ou un album. Le backend interroge les services musicaux externes et retourne des objets normalisés au frontend.

4.1.3 Lecture

Chaque titre peut être lancé depuis l'interface. Une attention particulière a été portée au comportement après connexion : l'application ne lance pas la musique automatiquement.

4.1.4 Favoris

Un utilisateur connecté peut sauvegarder un titre dans ses favoris et le retrouver plus tard. Les favoris sont persistés dans la base MySQL.

4.1.5 Playlists

L'utilisateur peut créer des playlists personnalisées, les renommer, les supprimer et y ajouter des chansons depuis le catalogue ou la bibliothèque. Les titres d'une playlist peuvent ensuite être écoutés, partagés ou retirés.

4.1.6 Partage

Chaque chanson possède un bouton de partage. Le lien généré contient la source et l'identifiant du titre. Exemple :

```
http://localhost:4200/?song=deezer:3135556
```

Lorsqu'un autre utilisateur ouvre ce lien, le titre est chargé après authentification.

4.1.7 Administration

Les comptes administrateurs peuvent consulter les statistiques, lister les utilisateurs, modifier les rôles et supprimer un utilisateur autre que leur propre compte.

4.2 Captures d'écran

Les captures suivantes ont été réalisées sur l'application locale. Elles présentent les principaux écrans développés : authentification, inscription, catalogue, recherche, partage, favoris, playlists, paramètres et administration.

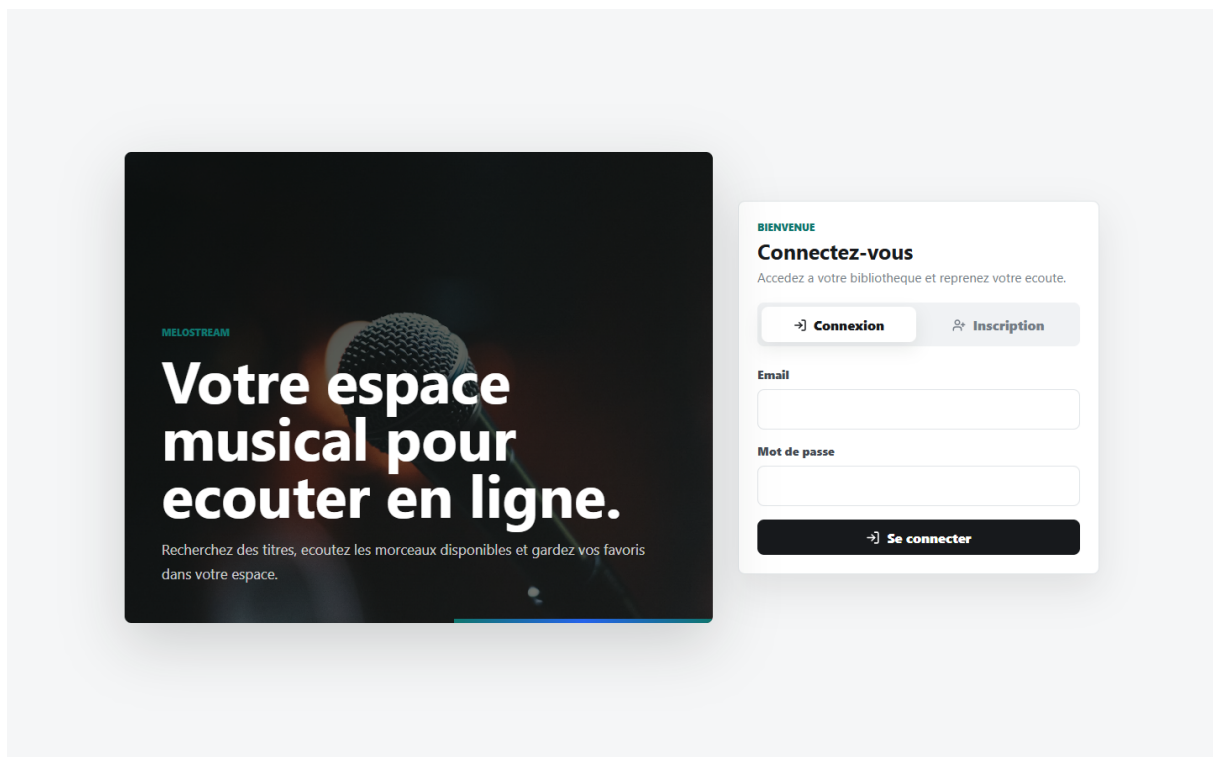


FIGURE 4.1 – Page d'authentification

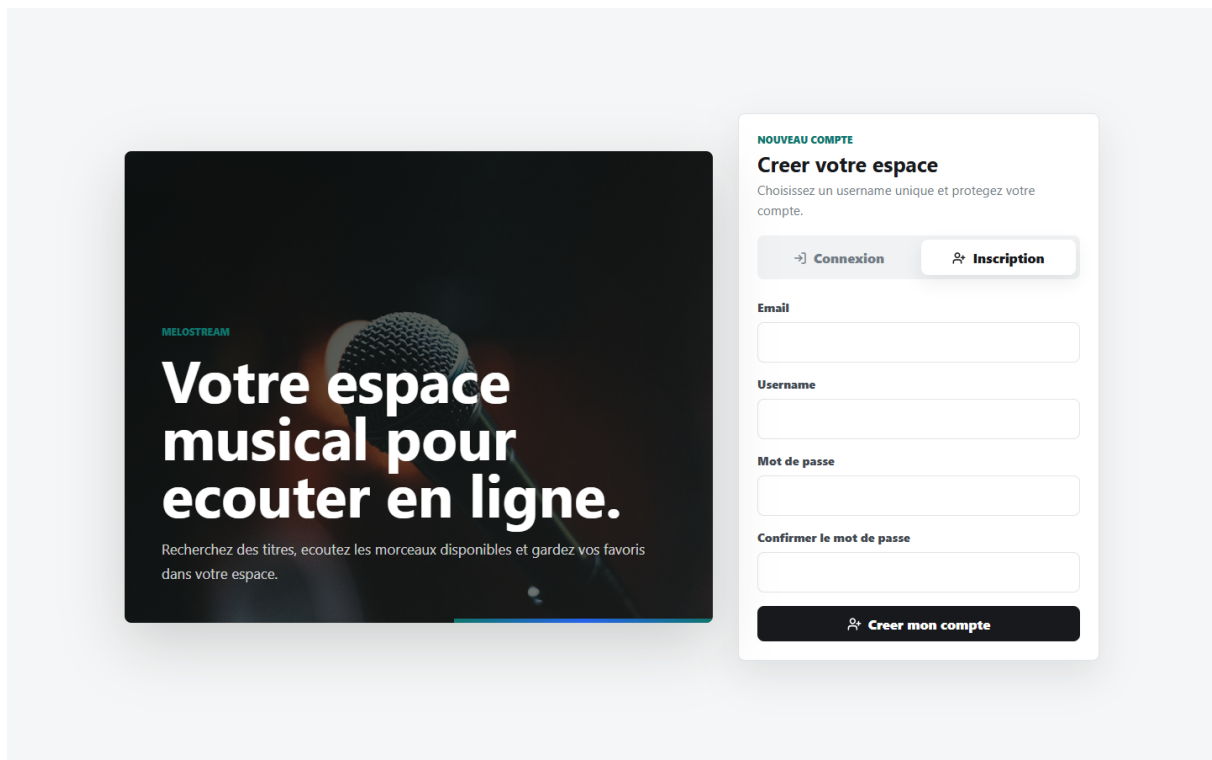


FIGURE 4.2 – Page d'inscription utilisateur

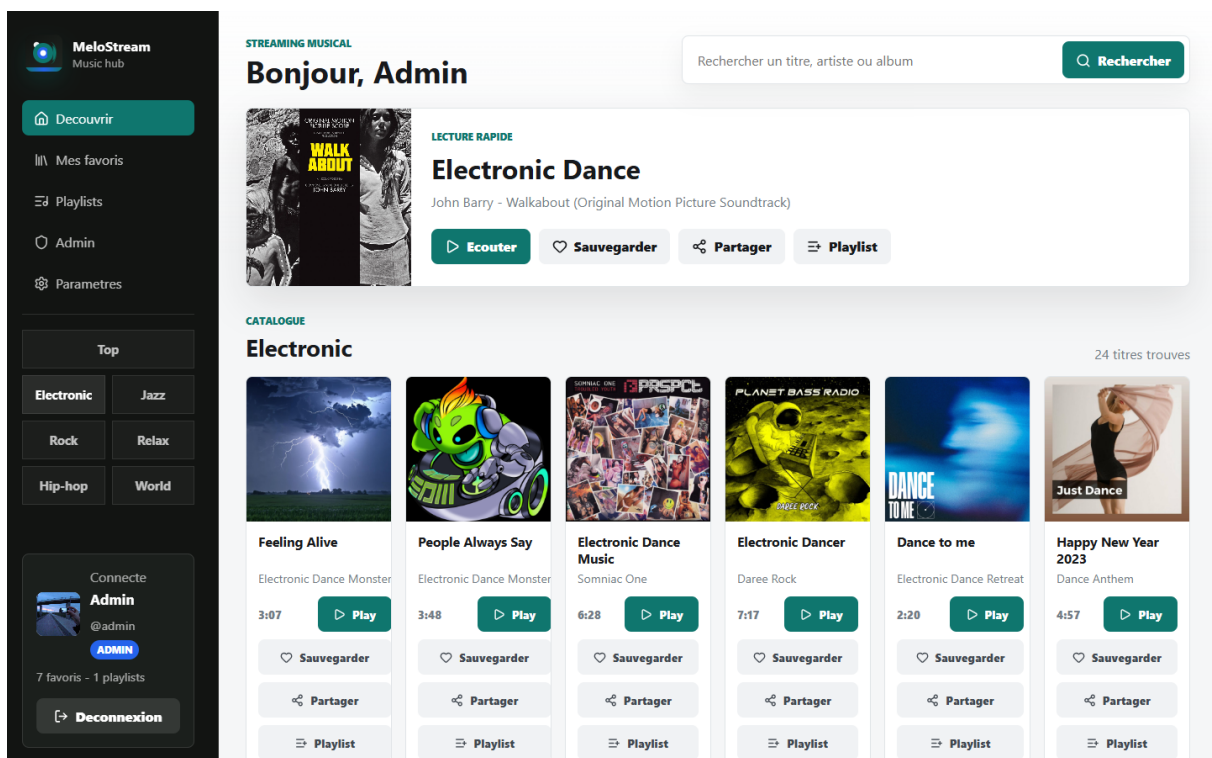


FIGURE 4.3 – Accueil utilisateur et catalogue musical

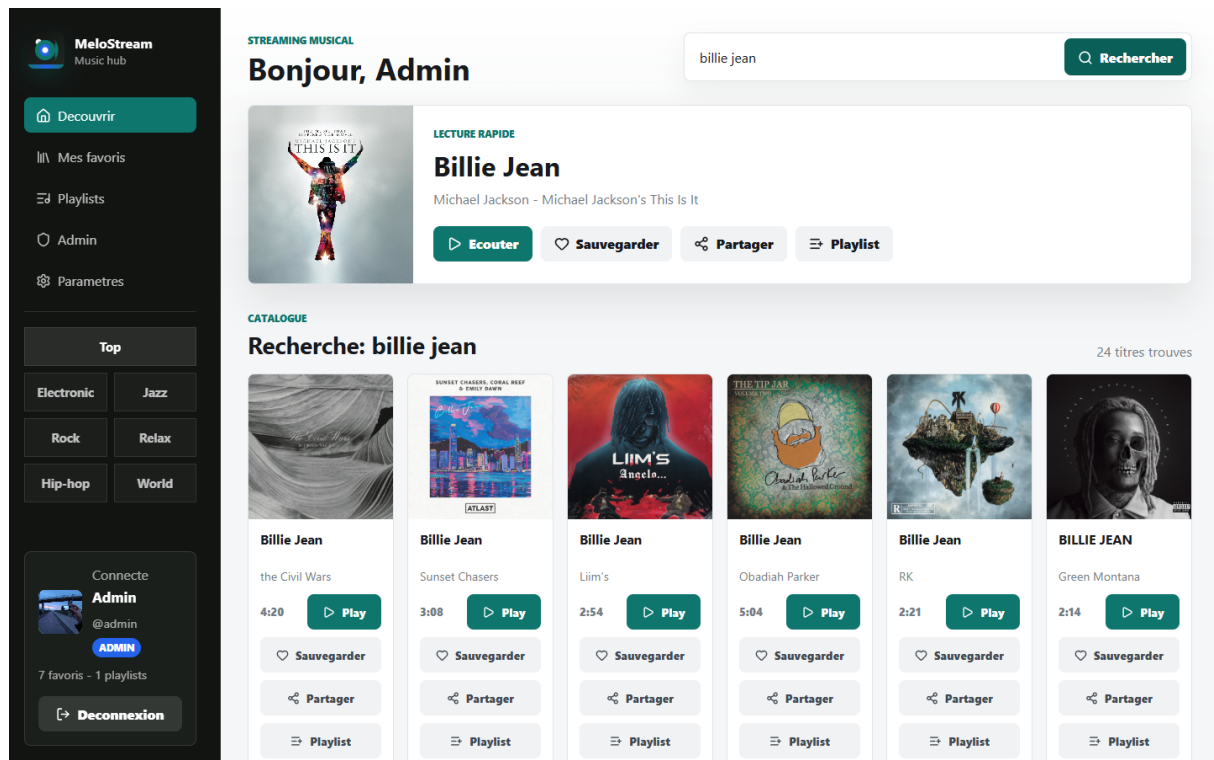


FIGURE 4.4 – Résultats d'une recherche de chanson

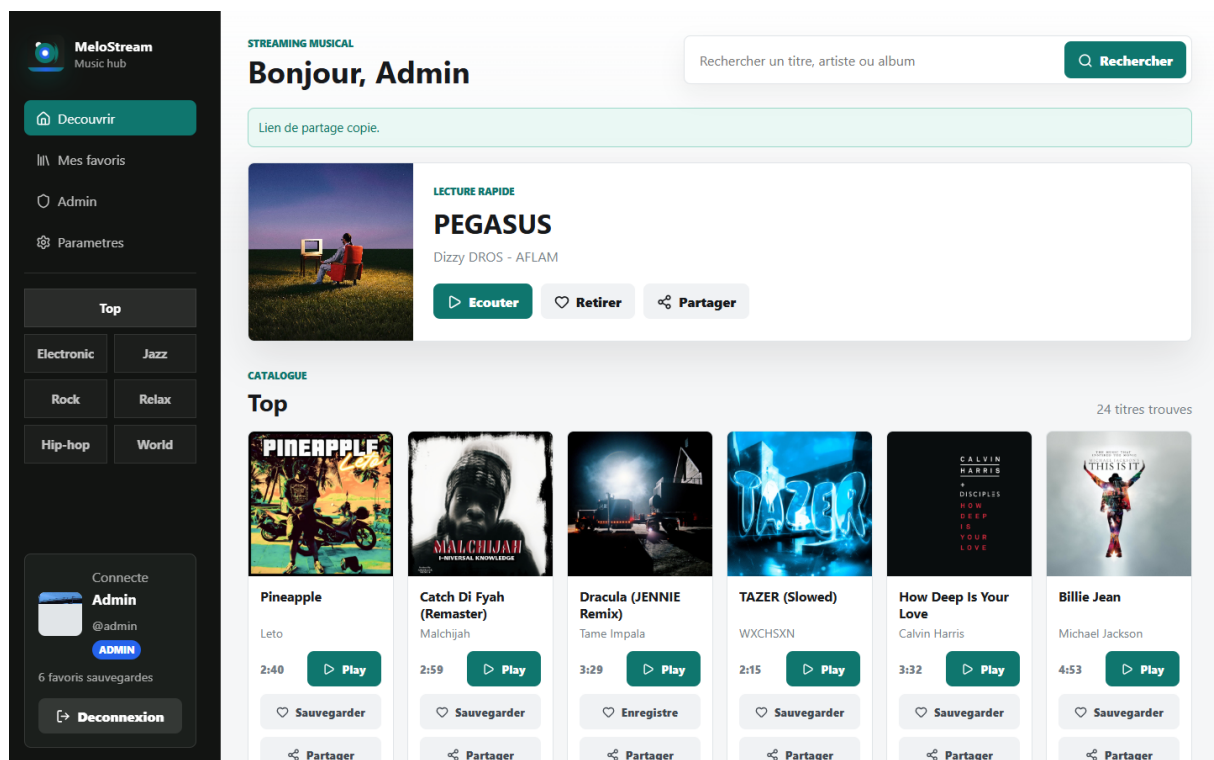


FIGURE 4.5 – Message affiché après le partage d'un titre

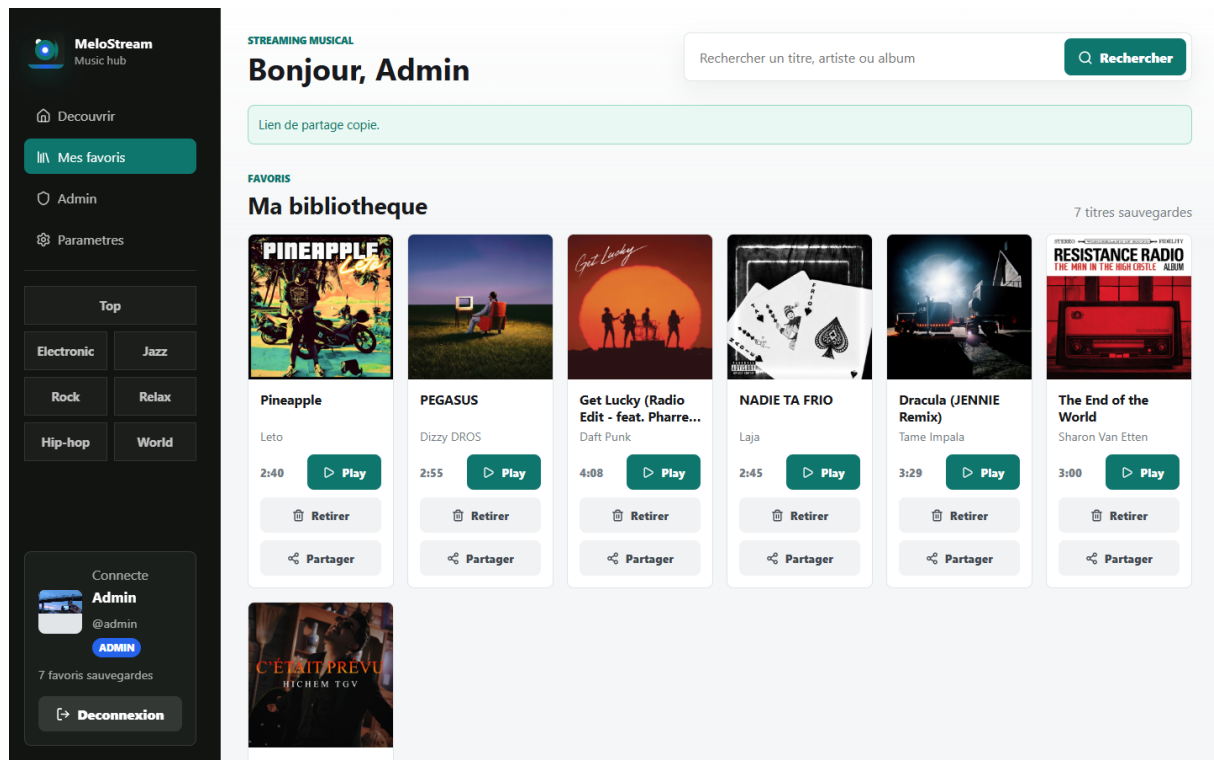


FIGURE 4.6 – Bibliothèque des titres favoris

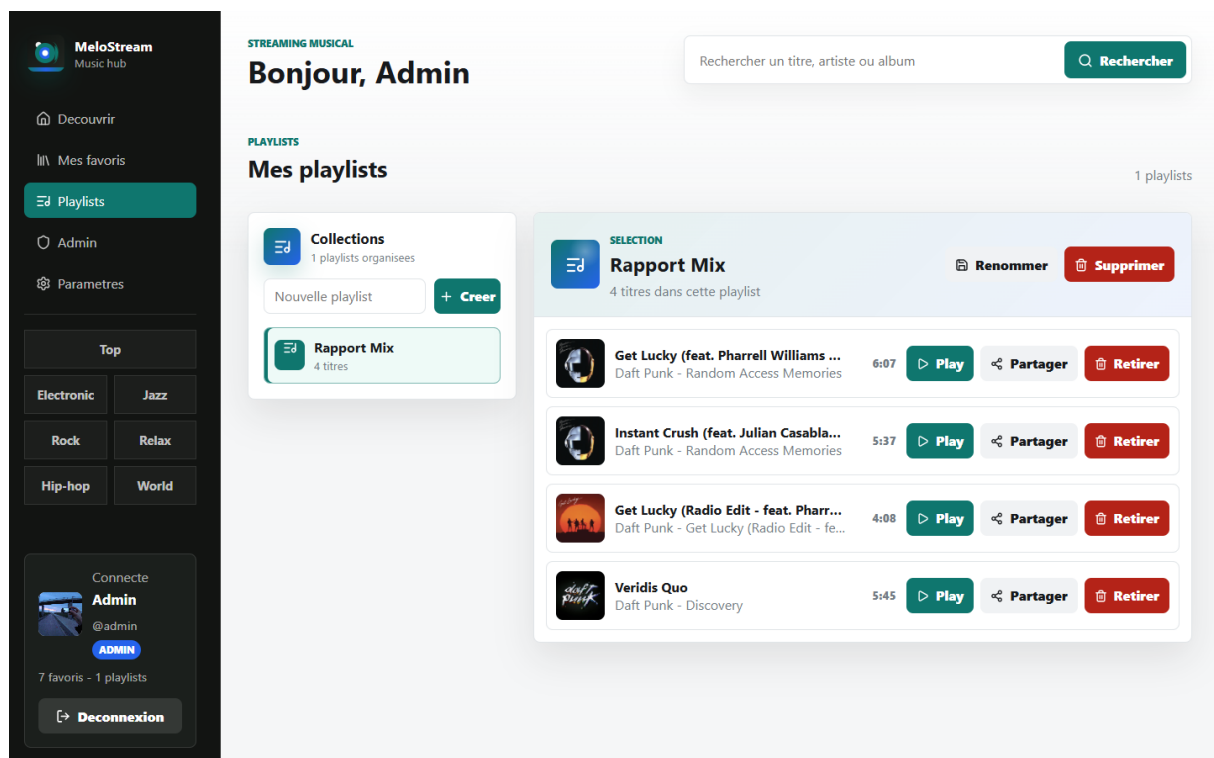


FIGURE 4.7 – Gestion des playlists utilisateur

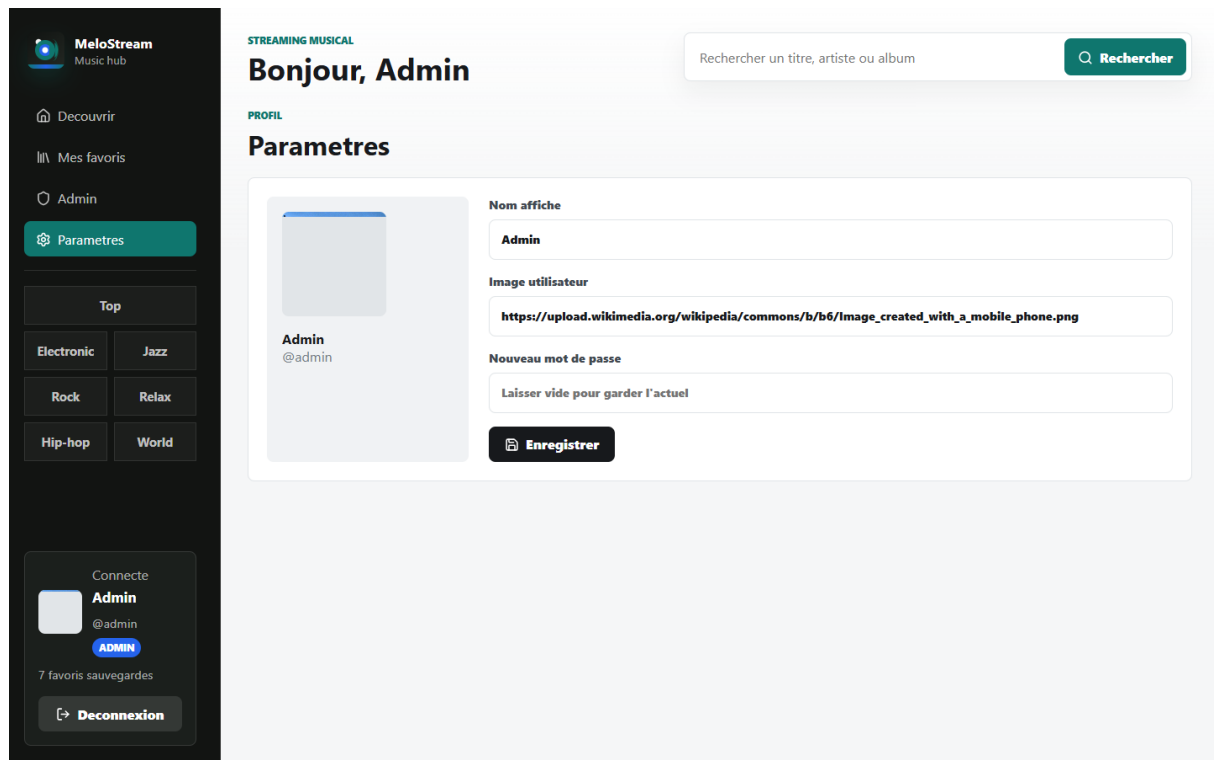


FIGURE 4.8 – Page des paramètres du profil

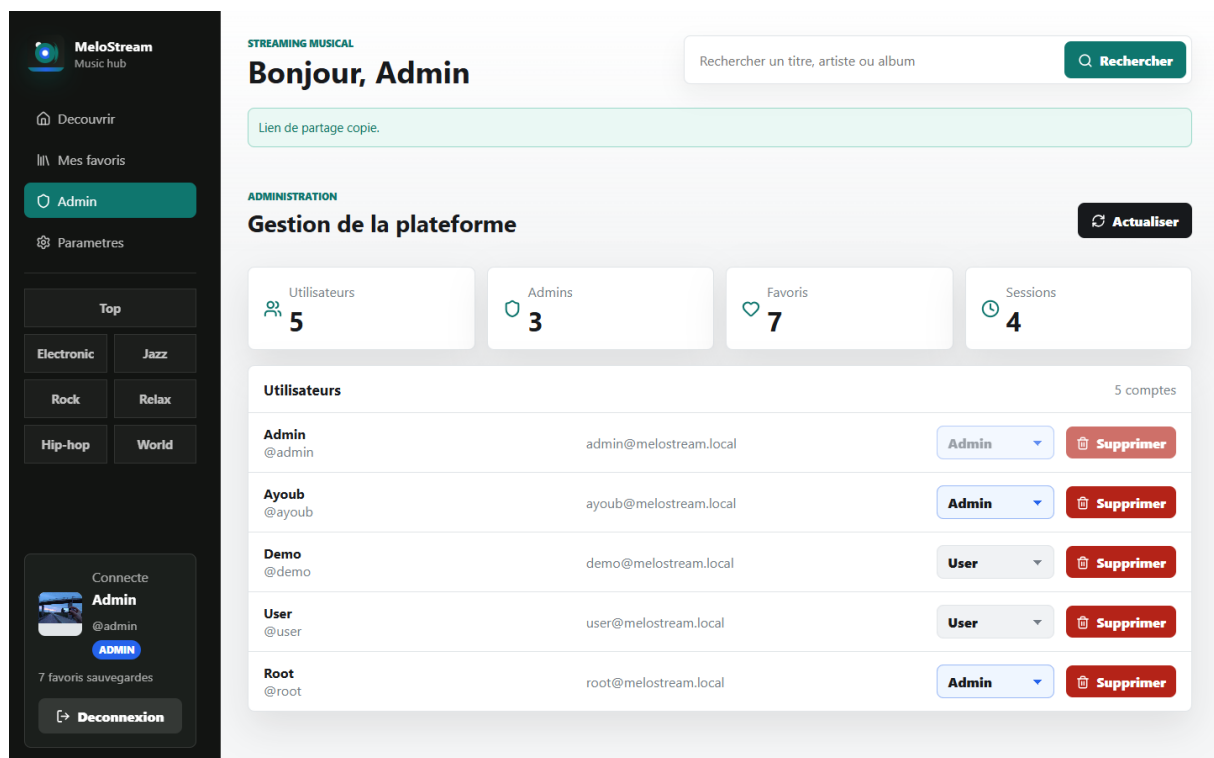


FIGURE 4.9 – Interface d'administration

4.3 Comptes de démonstration

Email	Mot de passe	Rôle
admin@melostream.local	admin123	ADMIN
ayoub@melostream.local	ayoub123	USER
demo@melostream.local	demo123	USER
user@melostream.local	user123	USER
root@melostream.local	root123	ADMIN

Chapitre 5

Installation et lancement

5.1 Prérequis

- Java 21.
- Node.js et npm.
- MySQL local.
- Maven Wrapper inclus dans le backend.

5.2 Configuration de la base

La configuration par défaut utilise MySQL en local :

```
database: music_stream
username: root
password: vide
```

La base est créée automatiquement grâce à l'option `createDatabaseIfNotExist=true`.

5.3 Lancement du backend

```
cd backend
.\mvnw.cmd spring-boot:run
```

Le backend démarre sur :

```
http://localhost:8080
```

5.4 Lancement du frontend

```
cd frontend
npm install
npm start
```

Le frontend démarre sur :

```
http://localhost:4200
```

5.5 Activation optionnelle de Jamendo

Pour utiliser Jamendo, il faut définir la variable d'environnement suivante avant de lancer le backend :

```
$env:JAMENDO_CLIENT_ID="ton_client_id_jamendo"
```

Chapitre 6

Vérification

6.1 Tests backend

```
cd backend  
.\mvnw.cmd test
```

6.2 Tests frontend

```
cd frontend  
npm test -- --watch=false
```

6.3 Build frontend

```
cd frontend  
npm run build
```

Conclusion

Le projet MeloStream répond aux objectifs fixés : il propose une plateforme musicale complète avec authentification, recherche, lecture, favoris, playlists, partage et administration. L'architecture sépare clairement le frontend Angular, le backend Spring Boot et la base MySQL, ce qui facilite la maintenance et les évolutions futures.

Les principaux défis rencontrés concernent l'intégration des API musicales externes, la gestion des liens de partage et la sécurisation des endpoints. Le projet peut encore être amélioré avec une route Angular dédiée au partage, une documentation OpenAPI, davantage de tests d'intégration et une configuration de déploiement adaptée à la production.